Artificial Intelligence Group Assignment Report

Ticket Routing Intelligence

Explainable Multilingual Queue and Priority Prediction for Customer Support Triage

36121 Artificial Intelligence Principles and Applications

AT3 – Group 2

Group member	Student ID
Tumadhir Alsharhan	25605852
Amal Maawadh	25605864
Andres Ethorimn	26029165
Laura Pacheco	25940875
Mario Rubiano	24900627
Carl Lazarus	12043485

Repository	https://github.com/ETHLOB/trn-uts-aipa
Prototype name	Ticket Routing Intelligence
Submission date	16 May 2026

Contents

1 Executive Summary **3**
- 1.1 Overview of the Project . . . 3
- 1.2 Objectives . . . 3
- 1.3 Key Findings . . . 3

2 Introduction **3**
- 2.1 Background and Context . . . 3
- 2.2 Problem Statement . . . 4
- 2.3 Significance and Impact . . . 4
- 2.4 Challenges and Constraints . . . 4
- 2.5 Project Objectives and Scope . . . 4

3 Theoretical Justification **4**
- 3.1 Overview of AI Methods Used . . . 4
- 3.2 Theoretical Foundations . . . 4
- 3.3 Justification of Selected Methods . . . 5
- 3.4 Transformer Benchmark Rationale . . . 5
- 3.5 Related Work / Literature Support . . . 5

4 Workflow and Methodology **5**
- 4.1 System Architecture Overview . . . 5
- 4.2 Data Collection and Preprocessing . . . 6
- 4.3 Feature Engineering . . . 6
- 4.4 Model Design and Algorithmic Approach . . . 6
- 4.5 Integration of AI Paradigms . . . 6
- 4.6 Design Decisions vs Implementation Details . . . 7

5 Empirical Analysis and Results **7**
- 5.1 Experimental Setup . . . 7
- 5.2 Dataset Description . . . 7
- 5.3 Evaluation Metrics . . . 9
- 5.4 Results . . . 10
- 5.5 Transformer Benchmark Results . . . 10
- 5.6 Result Interpretation and Discussion . . . 11

6 Critical Reflection **12**
- 6.1 Limitations of the Approach . . . 12
- 6.2 Failure Cases . . . 12
- 6.3 Trade-offs . . . 12
- 6.4 Scalability and Computational Constraints . . . 12
- 6.5 Alternative Methods and Improvements . . . 12

7 GitHub Repository **12**
- 7.1 Repository Link . . . 12
- 7.2 Instructions to Run the Project . . . 12

8 Conclusion **13**
- 8.1 Summary of Findings . . . 13
- 8.2 Key Insights . . . 13
- 8.3 Future Work . . . 13

9 Individual Contributions 13

10 References 15

1 Executive Summary

1.1 Overview of the Project

Ticket Routing Intelligence is an AI decision-support system for customer support ticket triage. The system analyses a customer ticket and returns five outputs: predicted queue/category, predicted priority, recommended support team, escalation flag, short summary, and explanation terms. The aim is not to replace human agents. The aim is to provide a structured first-pass recommendation so agents can review tickets faster and more consistently.

The implementation uses the Kaggle Multilingual Customer Support Tickets dataset. Five CSV files were audited. Three compatible multilingual files were merged, while two German-normalized files were excluded because their queue labels used a different taxonomy. After deduplication and filtering, the modelling dataset contains 44,275 usable tickets.

1.2 Objectives

The project objectives are:

- predict support queue/category from ticket text and safe pre-resolution metadata;
- predict priority as high, medium, or low;
- compare explainable NLP classifiers using validation and test metrics;
- benchmark multilingual transformer embeddings as an alternative representation;
- add rule-based routing and escalation logic;
- provide human-readable summaries and explanation terms;
- present the solution through a Streamlit dashboard with single-ticket and batch-ticket analysis.

1.3 Key Findings

The selected model is TF-IDF with Linear SVM for both prediction tasks. On the held-out test set, it reached macro F1 of 0.525 for queue/category prediction and 0.568 for priority prediction. Billing and Payments was the strongest category with F1 around 0.779, and high priority reached F1 around 0.624. These results are useful for first-pass triage, but they do not support full automation. This finding supports a human-in-the-loop design.

An optional sampled transformer-embedding benchmark was also implemented using multilingual MiniLM sentence embeddings and Logistic Regression. It reached sampled macro F1 of 0.235 for category and 0.350 for priority. In the current setup, transformer embeddings did not outperform the full-data TF-IDF Linear SVM pipeline. This result is useful because it shows that a more complex AI representation is not automatically better without tuning, larger samples, and careful validation.

2 Introduction

2.1 Background and Context

Customer support teams receive large volumes of unstructured tickets. These tickets can mention billing problems, product questions, account access, service outages, returns, security concerns, or general requests. In real operations, first-pass triage is important because it decides which team receives the ticket and how fast it is reviewed. Manual triage can be slow and inconsistent, especially when tickets are multilingual or when urgent words are hidden inside long messages.

2.2 Problem Statement

The problem is slow and inconsistent ticket triage. A support ticket must be mapped to a queue, assigned a priority, checked for escalation risk, and routed to the correct team. If the first decision is wrong, customers wait longer and urgent cases can be missed. This project builds an AI prototype that assists this decision while keeping humans responsible for final action.

2.3 Significance and Impact

The system can reduce manual work by giving agents an initial recommendation. It can also improve transparency by showing explanation terms and escalation reasons. This is important because a support team should not rely on an unexplained prediction. The prototype also supports batch upload, so a team can analyse several tickets from a CSV or Excel file.

2.4 Challenges and Constraints

The main constraints are dataset imbalance, multilingual text, overlapping category language, and privacy. Some categories share similar words, such as product support and technical support. Some languages have fewer examples, which affects fairness. Also, ticket text may include personal information, so the summary function masks obvious email addresses and phone numbers. Finally, the project must stay reproducible for an academic assignment, so the deployed demo uses simple and explainable methods instead of external LLM services.

2.5 Project Objectives and Scope

The project scope is a working prototype and reproducible pipeline. It includes notebooks, reusable Python modules, saved metrics, charts, and a Streamlit app. It does not claim production readiness. A real deployment would need ticketing-system integration, access control, monitoring, retraining workflows, stronger privacy redaction, and testing on company-specific data.

3 Theoretical Justification

3.1 Overview of AI Methods Used

The solution combines several AI methods:

- supervised NLP classification for queue and priority prediction;
- TF-IDF vectorisation to convert text into sparse numerical features;
- Logistic Regression, Naive Bayes, and Linear SVM as comparison models;
- rule-based reasoning for routing and escalation;
- template-based summarisation with privacy masking;
- explanation terms from influential TF-IDF features;
- optional multilingual transformer embeddings for comparison evidence.

3.2 Theoretical Foundations

TF-IDF gives high weight to words that are frequent in one document but less frequent across the full corpus. This is useful for support tickets because terms such as “invoice”, “refund”, “outage”, or “password” can be strong signals for a queue. Linear SVM is suitable for sparse, high-dimensional text features because it learns a separating boundary between classes and often performs well for

text classification. Logistic Regression gives a strong linear baseline with probabilistic interpretation, while Naive Bayes is a simple probabilistic baseline that is often used for text.

The rule-based layer represents operational knowledge. For example, if a ticket includes words such as “unauthorised”, “fraud”, or “breach”, the app can recommend a security team or escalation review even if the predicted queue is not security. This combination is important because ML predictions and business rules solve different parts of the triage task.

3.3 Justification of Selected Methods

Linear SVM was selected because it had the best validation macro F1 for both queue and priority. It also keeps the system lightweight, fast, and explainable. The app can show influential TF-IDF terms, which helps agents understand the prediction. This approach is more suitable for the current prototype than a heavier model that is harder to run and explain.

The transformer benchmark was kept separate from live prediction. It provides a semantic multilingual comparison, but it used a sample of 2,500 tickets and performed below the current model. Therefore, it is report evidence and future-work evidence, not the deployed prediction path.

3.4 Transformer Benchmark Rationale

The transformer benchmark used the multilingual MiniLM sentence-transformer model to encode ticket text as dense multilingual sentence embeddings. A Logistic Regression classifier was then trained on those embeddings for the same two tasks: category prediction and priority prediction. This experiment was added because transformer embeddings are designed to capture semantic similarity across languages, which is relevant for a multilingual ticket dataset. However, it was intentionally run as a sampled benchmark because transformer inference is heavier than TF-IDF and was not required for the live demo.

3.5 Related Work / Literature Support

The project follows common NLP classification practice: text normalisation, TF-IDF feature extraction, train/validation/test splitting, model comparison, and held-out evaluation. The responsible AI design follows principles of human oversight, leakage control, transparency, and privacy minimisation.

4 Workflow and Methodology

4.1 System Architecture Overview

The workflow has seven main stages: audit data, prepare text, build features, train models, evaluate results, route tickets, and explain outputs. The Streamlit app presents this workflow in the Overview tab and provides prediction tools in the Try Solution tab.

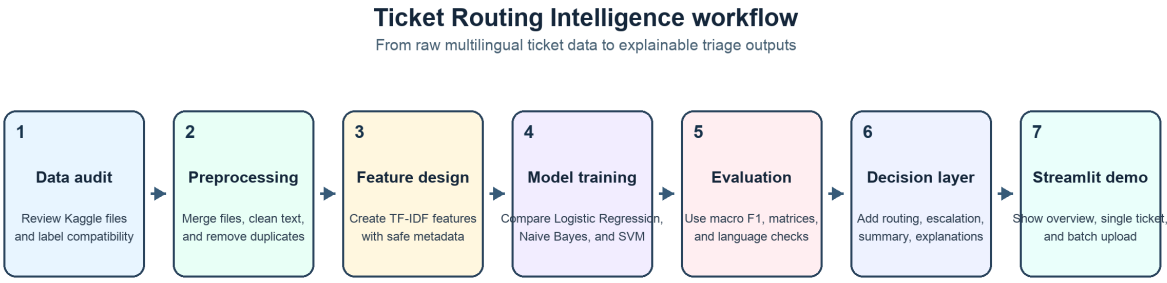


Figure 1: Project workflow used in the final prototype.

4.2 Data Collection and Preprocessing

The dataset was downloaded from Kaggle. The compatible files were:

- `aa_dataset-tickets-multi-lang-5-2-50-version.csv`
- `dataset-tickets-multi-lang-4-20k.csv`
- `dataset-tickets-multi-lang3-4k.csv`

The two German-normalized files were audited but excluded from the main supervised model because their queue/category taxonomy was different.

The pipeline normalises column names, combines subject and body into `ticket_text`, removes duplicate subject-body pairs, and removes rows without usable text. It excludes the `answer` field because this field is written after a support agent responds. Using it would create target leakage because the information would not be available at ticket creation time.

4.3 Feature Engineering

The main feature is `model_text`. It combines cleaned ticket text with safe metadata tokens such as type, language, and tags. The cleaning process lowercases text, masks emails and phone numbers, replaces long numbers, removes URLs, and keeps simple alphanumeric tokens. TF-IDF uses unigrams and bigrams with a maximum of 30,000 features.

4.4 Model Design and Algorithmic Approach

The project trains two separate supervised tasks:

- queue/category prediction using the dataset queue label;
- priority prediction using the dataset priority label.

For each task, three models were compared: Logistic Regression, Multinomial Naive Bayes, and Linear SVM. The data was split into training, validation, and test sets. Candidate models were selected using validation macro F1. The selected model was retrained on training plus validation data, then evaluated on the held-out test set.

4.5 Integration of AI Paradigms

The final prototype integrates supervised learning, probabilistic baseline modelling, symbolic rules, explainability, template summarisation, and transformer comparison. This satisfies the requirement to show more than one AI paradigm. However, only the stable and explainable TF-IDF Linear SVM pipeline is used for live predictions.

4.6 Design Decisions vs Implementation Details

A key design decision was to prioritise reproducibility and explainability. The app does not call an external LLM. The summary is template-based, and the explanation is based on TF-IDF terms. This choice reduces cost, privacy risk, and dependency risk. Another decision was to display transformer benchmark results in the app Overview tab, but not to use them for routing.

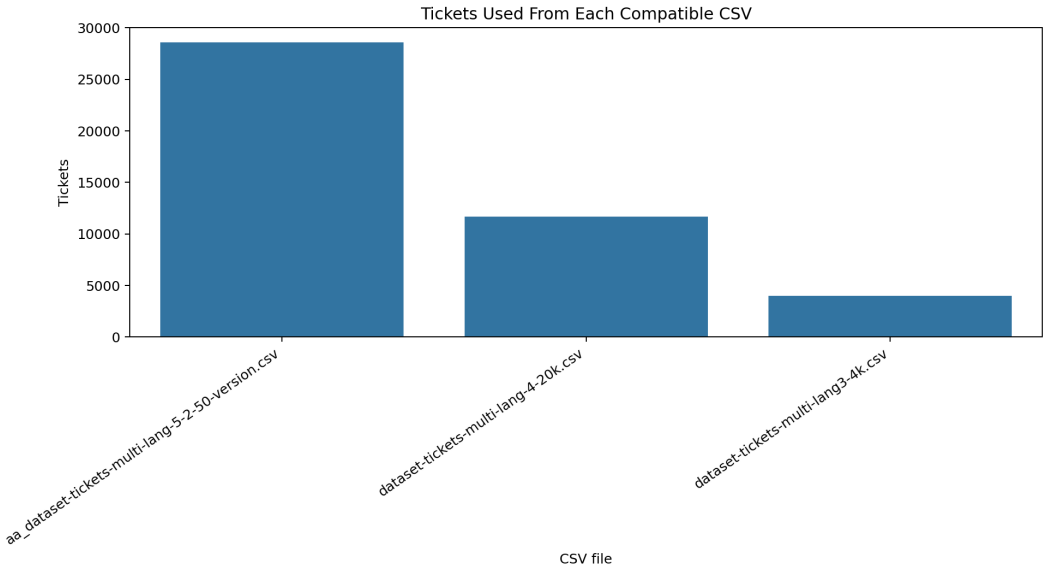
5 Empirical Analysis and Results

5.1 Experimental Setup

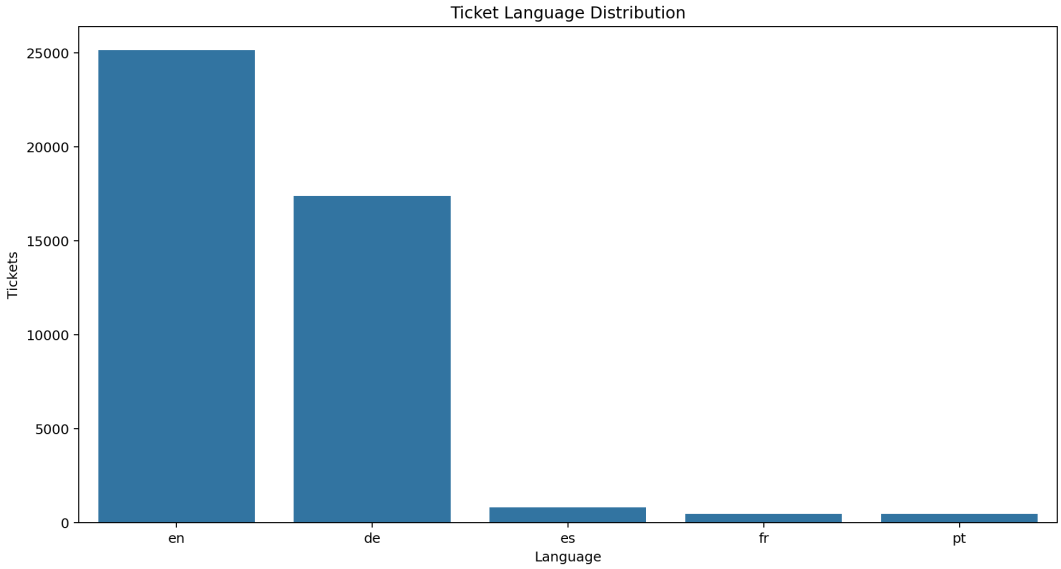
The final modelling frame contains 44,275 rows. The pipeline uses a train/validation/test approach with stratification when possible. The same random seed is used for reproducibility. Metrics are saved in the `results/` folder and charts are saved in `report_assets/`.

5.2 Dataset Description

The data includes multilingual support tickets across English, German, Spanish, French, and Portuguese. English and German dominate the dataset. The dataset includes multiple support queues, including Billing and Payments, Product Support, Technical Support, Customer Service, Service Outages and Maintenance, Returns and Exchanges, Sales and Pre-Sales, Human Resources, and General Inquiry.

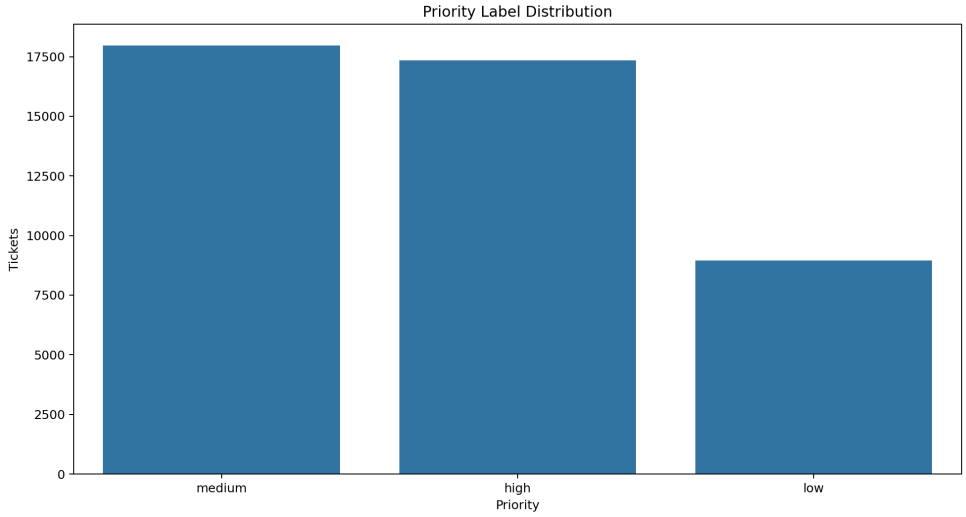


(a) Rows contributed by each compatible Kaggle CSV file.

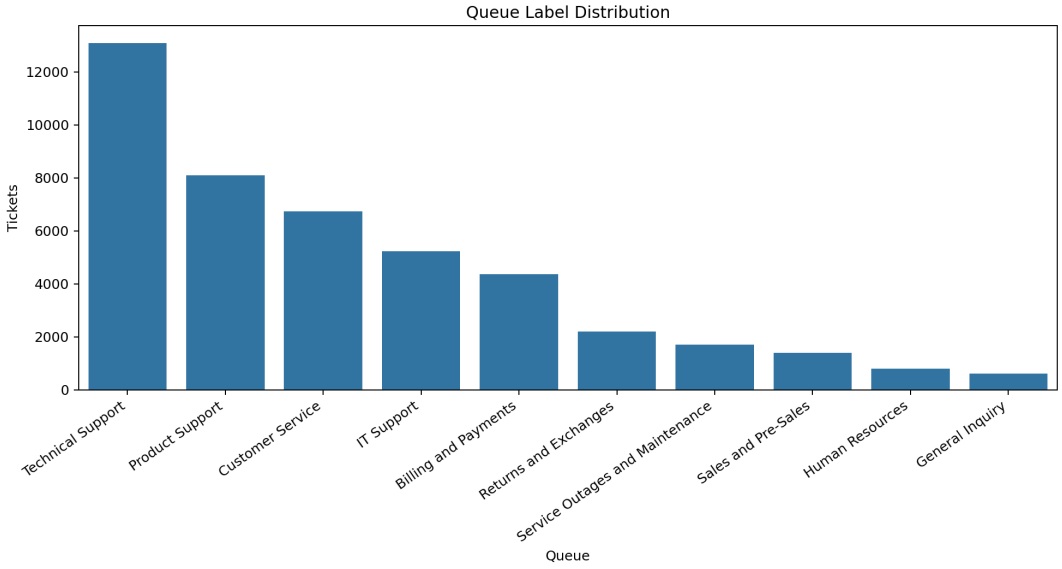


(b) Ticket language distribution after preprocessing.

Figure 2: Dataset composition used by the final modelling pipeline.



(a) Priority distribution.



(b) Queue/category distribution.

Figure 3: Target-label distributions for priority and queue prediction.

5.3 Evaluation Metrics

The project reports accuracy, macro precision, macro recall, macro F1, and weighted F1. Macro F1 is the main model-selection metric because it gives equal importance to each class. This is important for imbalanced datasets because a model should not only optimise performance on majority classes.

5.4 Results

Task	Model	Split	Accuracy	Macro P	Macro R	Macro F1
Category	Linear SVM	Test	0.531	0.507	0.547	0.525
Priority	Linear SVM	Test	0.581	0.567	0.569	0.568
Category	Transformer + LR	Sampled	0.272	0.249	0.293	0.235
Priority	Transformer + LR	Sampled	0.357	0.359	0.359	0.350

Table 1: Main test results and optional transformer benchmark results.

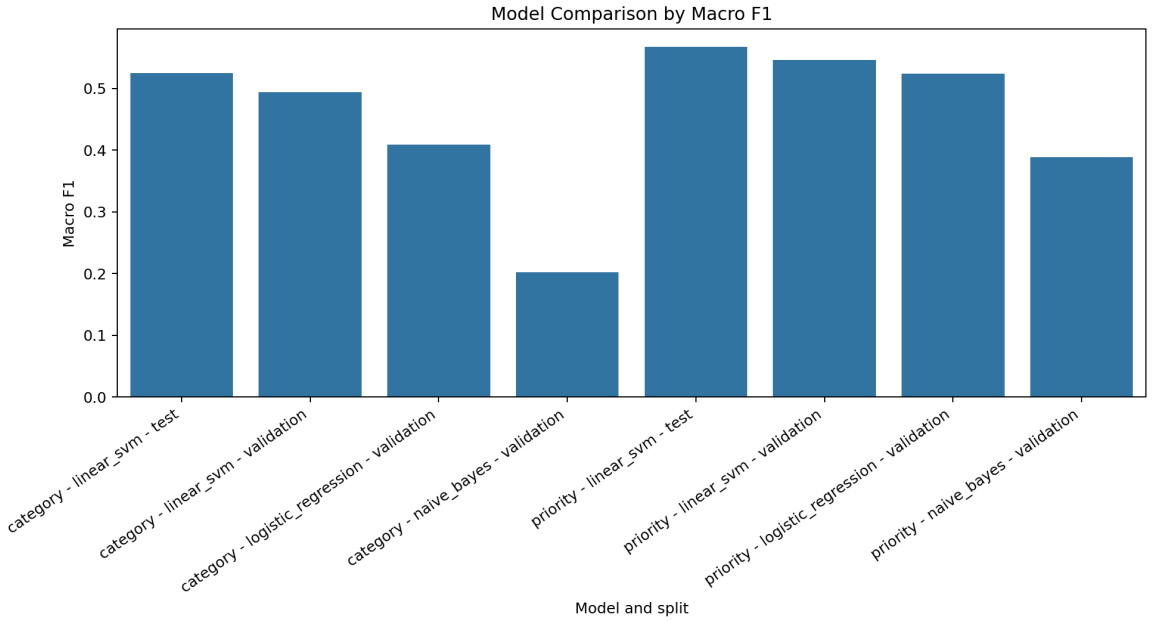


Figure 4: Validation and test macro F1 comparison for the main models.

5.5 Transformer Benchmark Results

The optional transformer benchmark was run on a reproducible sample of 2,500 tickets. The dense embeddings were generated with multilingual MiniLM and classified with Logistic Regression. The category model reached 0.235 sampled macro F1, and the priority model reached 0.350 sampled macro F1. These results were lower than the full-data TF-IDF Linear SVM results: 0.525 macro F1 for category and 0.568 macro F1 for priority.

Task	Representation and classifier	Macro F1	Accuracy
Category	TF-IDF + Linear SVM, full test set	0.525	0.531
Category	Multilingual MiniLM embeddings + Logistic Regression, sampled test	0.235	0.272
Priority	TF-IDF + Linear SVM, full test set	0.568	0.581
Priority	Multilingual MiniLM embeddings + Logistic Regression, sampled test	0.350	0.357

Table 2: Direct report comparison between the deployed TF-IDF model and the optional transformer benchmark.

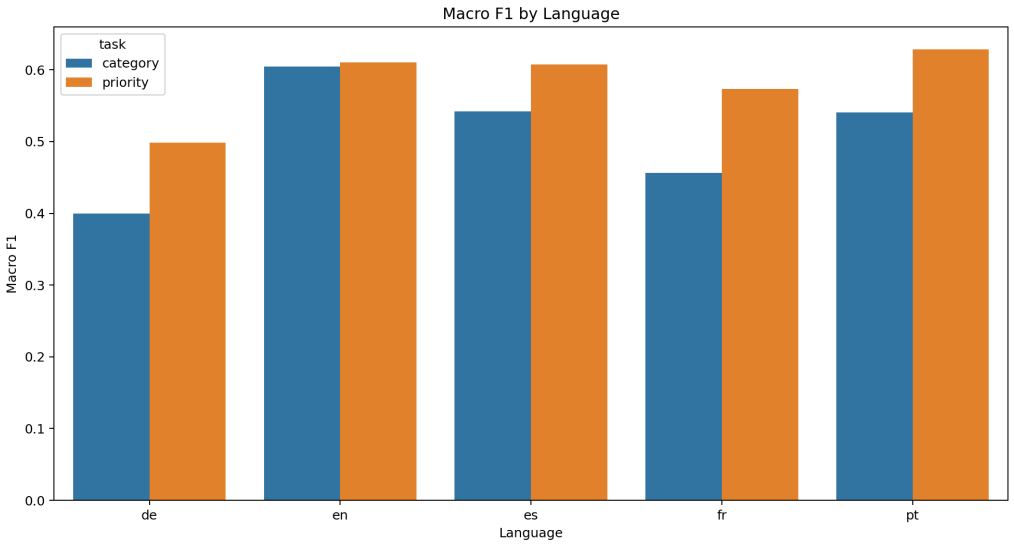
The transformer result does not mean that transformers are unsuitable for this problem. It means that this specific sampled and lightly tuned experiment did not outperform the simpler pipeline. A stronger transformer setup would need more data, careful class balancing, hyperparameter tuning,

and possibly a classifier better matched to embeddings. For the current assignment, the result supports a responsible technical choice: keep Linear SVM for live prediction and discuss transformer embeddings as validated future work.

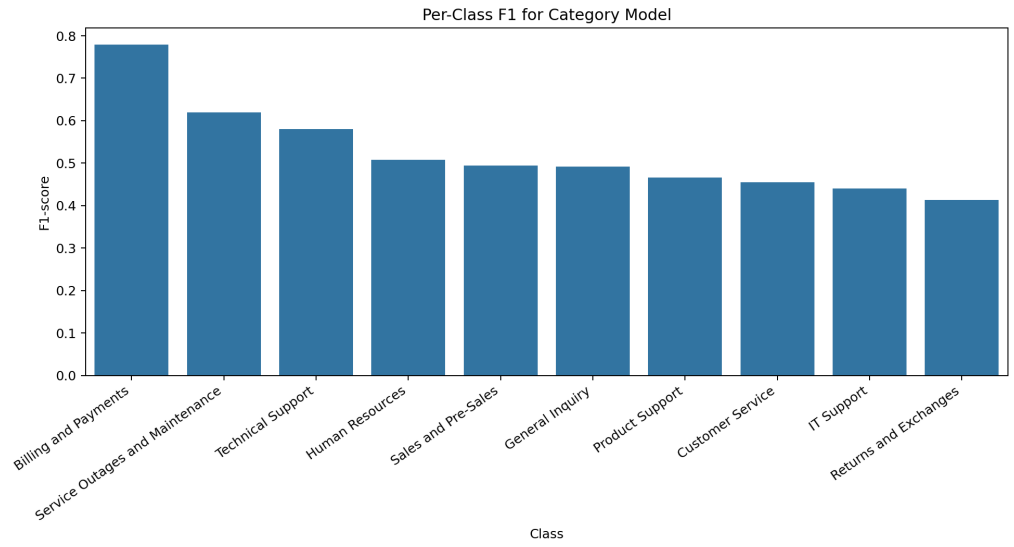
5.6 Result Interpretation and Discussion

The results show useful predictive signal but also important limitations. Billing and Payments is the strongest category because billing tickets have distinctive vocabulary. Service Outages and Maintenance also performs relatively well. Product Support, IT Support, and Customer Service are harder because their wording overlaps. Priority prediction performs slightly better than category prediction, but low priority remains weaker than high and medium.

Language analysis shows a performance gap. English category macro F1 is 0.604, while German category macro F1 is 0.400. This suggests that future work should improve multilingual preprocessing, use more balanced training data, or tune multilingual embeddings more carefully.



(a) Macro F1 by language and task.



(b) Per-class category F1.

Figure 5: Detailed performance views used for critical reflection.

6 Critical Reflection

6.1 Limitations of the Approach

The first limitation is moderate macro F1. The model is useful for support, but not for automatic final decisions. The second limitation is language imbalance. The third limitation is the training/inference mismatch: training uses text plus safe metadata, while a manually pasted ticket in the app mainly provides text. Batch files can include richer columns, but single-ticket mode is simpler. The fourth limitation is privacy masking. The app masks obvious emails and phone numbers, but it does not detect all personal information.

6.2 Failure Cases

Some tickets are ambiguous. A security-related issue may still mention billing details and be predicted as Billing and Payments. In that case, the model queue and the recommended team can be different. This is expected because queue prediction is learned from labels, while team recommendation also uses security terms. This is a useful design feature, but it must be explained to users.

6.3 Trade-offs

The main trade-off is explainability versus complexity. TF-IDF Linear SVM is simple, fast, and explainable. Transformer embeddings can capture deeper semantic meaning, but in this sampled benchmark they performed worse and required heavier dependencies. LLM summarisation could create more natural summaries, but it would add privacy, cost, and reproducibility concerns.

6.4 Scalability and Computational Constraints

The current prototype runs locally and uses saved model files. A production version would need an API, authentication, logging, monitoring, drift detection, retraining, and integration with a ticketing platform. The transformer benchmark was limited by sample size and compute. Larger experiments may need better hardware or cloud resources.

6.5 Alternative Methods and Improvements

Future work should include confidence calibration, active learning from agent corrections, Unicode-aware multilingual preprocessing, stronger PII redaction, SLA-aware escalation, and better multilingual transformer experiments. A text-only model could also reduce the training/inference mismatch for single-ticket mode.

7 GitHub Repository

7.1 Repository Link

The GitHub repository is:

https://github.com/ETHLOB/trn-uts-aipa

7.2 Instructions to Run the Project

Create and install the environment:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip install -e .
```

Download the Kaggle files:

```
.\.venv\Scripts\python.exe -m customer_support_ai.download_data
```

Train and evaluate the models:

```
.\.venv\Scripts\python.exe -m customer_support_ai.train
.\.venv\Scripts\python.exe -m customer_support_ai.report_assets
```

Run the Streamlit app:

```
.\.venv\Scripts\streamlit.exe run app/streamlit_app.py
```

Optional transformer benchmark:

```
.\.venv\Scripts\python.exe -m pip install -r requirements-transformer.txt
.\.venv\Scripts\python.exe -m customer_support_ai.transformer_benchmark --sample-size
↪  2500
```

8 Conclusion

8.1 Summary of Findings

This project built a working AI decision-support prototype for customer support triage. It uses a real multilingual dataset, reproducible preprocessing, supervised model comparison, held-out evaluation, routing rules, escalation logic, explanation terms, and a Streamlit dashboard.

8.2 Key Insights

The strongest current approach is TF-IDF with Linear SVM. It gives moderate but useful performance and supports explainability. The results are strongest where categories have clear vocabulary and weaker where categories overlap or language representation is limited. The optional transformer benchmark did not improve performance in the current setup, so the final design keeps the simpler model for live predictions.

8.3 Future Work

Future work should improve multilingual handling, add confidence scores, learn from human corrections, strengthen privacy redaction, and test more transformer or neural methods under fairer conditions. Production deployment would also need monitoring, data governance, and integration with support systems.

9 Individual Contributions

The table below summarises the individual contributions using repository history, presentation artefacts, and the WhatsApp discussion. Contributions that were discussed but are not present in the current main repository are marked as pending evidence.

Member	Confirmed contribution in repository or local artefacts	Pending contribution evidence
Tumadhir Alsharhan ID: 25605852	Led the development of the main customer support AI pipeline, including notebooks 01–05, reusable Python modules, Streamlit app, README, run checklist, result files, charts, and demo examples. Amal Maawadh supported these tasks through project planning and presentation/report framing. Tumadhir also prepared and updated group presentation material and Q&A guidance.	Confirmed in repository history, WhatsApp evidence, and presentation artefacts.
Mario Rubiano ID: 24900627	Reviewed the codebase, notebooks, app, outputs, and assignment requirements. Added executed notebook evidence and a status/gap report. Implemented the optional transformer-embedding benchmark, result files, notebook 06, transformer dependency file, README/checklist updates, and Streamlit display of transformer benchmark results. Prepared the Speaker 6 responsible AI and future work content.	Confirmed in repository history and presentation artefacts.
Andres Ethorimn ID: 26029165	Created the original repository/template, shared GitHub access, maintained the main branch, merged pull requests, cleaned data and presentation files from main, restored result artefacts, and reviewed Mario's PR before adding features to main. Presented empirical results in the slide notes.	Confirmed in repository history, WhatsApp evidence, and presentation artefacts.
Amal Maawadh ID: 25605864	Supported Tumadhir in the main project planning and implementation direction. Shared a detailed implementation plan covering the dataset, AI workflow, models, evaluation metrics, demo features, and HD-level requirements. Drafted presentation content and speaker notes. Presentation artefacts show Amal as Speaker 1 for the project introduction and problem framing.	Confirmed by WhatsApp evidence and presentation artefacts.
Laura Pacheco ID: 25940875	Presentation artefacts show Laura as Speaker 2 for the solution overview, dataset explanation, and leakage-control explanation. WhatsApp evidence also indicates that Laura explored EDA responsibilities and later experimented with an MLP priority model and an RL routing agent.	**PENDING:** Laura Pacheco: the MLP Neural Network and RL Routing Agent are mentioned in chat, but no corresponding code or results are present in current main repository content. Add branch output or keep this item as future/pending work.

Member	Confirmed contribution in repository or local artefacts	Pending contribution evidence
Carl Lazarus ID: 12043485	Presentation artefacts show Carl as Speaker 3 for the AI methods section. WhatsApp evidence also indicates that Carl added a demo slide/video and refined the demo wording for the presentation.	**PENDING:** Carl Lazarus: the LLM AI assistant for ticket submission is mentioned as branch work in chat, but no corresponding code is present in current main repository content. Add branch output or keep this item as future/pending work.

10 References

1. Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273–297.
2. Joachims, T. (1998). Text categorization with support vector machines: learning with many relevant features. *ECML*.
3. McCallum, A. and Nigam, K. (1998). A comparison of event models for Naive Bayes text classification. *AAAI Workshop on Learning for Text Categorization*.
4. Ramos, J. (2003). Using TF-IDF to determine word relevance in document queries. *Proceedings of the First Instructional Conference on Machine Learning*.
5. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP-IJCNLP*.
6. scikit-learn developers. (2026). scikit-learn: Machine Learning in Python. https://scikit-learn.org/
7. Streamlit developers. (2026). Streamlit documentation. https://docs.streamlit.io/
8. Kaggle. (2026). Multilingual Customer Support Tickets dataset. https://www.kaggle.com/datasets/tobiasbueck/multilingual-customer-support-tickets